Fondamenti di programmazione

Canale A-L - Prof. M. Mancini

AA 26-27 - Lezione 5

RECAP lezione 4:

- contenitori
- assegnazione a slice
- metodi dei contenitori

In [ ]:

```
# Soluzione
L = list(range(1, 22, 7))         # [ 1 8 15 ]            in questo ordine
S = set(range(0, -57, -14))       # { 0 -14 -28 -42 -56 } in qualsiasi ordine
L[1:1] = S                        # inserisco all'elemento di indice 1 (8) gli elementi dell'insieme S
print(L)
print(S)
```

Moduli e import

Per caricare **nel namespace GLOBALE** del programma principale le funzionalità realizzate in altri file si usa **`import`**

```
import modulo                  # solo il *nome* del modulo viene inserito nel namespace
modulo.funzione(argomenti)     # come chiamare una funzione del modulo
A = modulo.variabile           # come leggere una variabile del modulo
```

```
from modulo import funzione  # inserisco solo il nome 'funzione' nel namespace
funzione(argomenti)          # come chiamare la funzione
```

NOTA: potrei avere una collisione con un'altra funzione con lo stesso nome

Altrimenti posso importare TUTTI i nomi del modulo nel namespace globale

```
from modulo import *       # BAD CODER!!! NON USATELO!!!
funzione(argomenti)        # come chiamare la funzione
```

NOTA: potrei avere una collisione con altri nomi già definiti

NOTA TECNICA

- A loro volta gli altri moduli possono importare altre librerie, nel proprio namespace di modulo
- Se un modulo è stato già importato **viene riusato senza rileggerlo di nuovo** dal file

In [ ]:

```
# esempio: importo il modulo 'math' e stampo pi greco
import math           # importando nel namespage globale solo il modulo
print(math.pi)
# help(math)          # per sapere le funzioni disponibili in math

from math import pi  # importando nel namespage globale solo pi
print(pi)
```

Funzioni: come definire parti di programma riusabili

- **MAI:** copiare e incollare 2 o più volte lo stesso pezzo di codice ( )
- **SEMPRE:** definisci una funzione separata e chiamala quante volte vuoi

PERCHE'? per ogni correzione andranno corrette tutte quante le copie (che strazio!!!)

Sintassi: (come si scrive)

```
def nome_della_funzione( argomenti ):
    '''docstring che descrive cosa fa
       anche su più righe'''                # sempre consigliata, genera l'help
    istruzioni che calcolano il risultato
    return risultato
```

Il corpo della funzione DEVE essere indentato (in genere di 4 spazi) cioè allineato più a destra della parola **def**
Le istruzioni indentate allo stesso livello formano un blocco di istruzioni da eseguire in sequenza
NON SI USA il carattere TAB (`\t`) ma spazi
Gli IDE inseriscono 4 spazi quando battete il tasto TAB
Gli editor di puro testo non è detto che lo facciano

Gli `argomenti` tra parentesi tonde sono i nomi che **indicano le informazioni necessarie**,
da fornire in modo che la funzione possa svolgere il suo compito
NOTA: se definite nuove variabili dentro il CORPO della funzione (la parte indentata) i loro nomi sono disponibili SOLO all'interno della funzione
Il `risultato` è il dato che la funzione ha calcolato e che viene fornito al programma che l'ha chiamata con la istruzione **`return`**

ESEMPIO: calcoliamo l'altezza media, minima e massima di un gruppo di persone

La **media** non è altro che la somma delle altezze diviso per il numero di altezze

In [ ]:

```
# come si definisce la funzione
def media_altezze(altezze):
    'calcolo la media di un gruppo di altezze'   # docstring
    min_altezze = min(altezze)                   # uso la funzione 'min' di sistema
    max_altezze = max(altezze)                   # uso la funzione 'max' di sistema
    somma_altezze = sum(altezze)                 # uso la funzione 'sum' di sistema
    media = somma_altezze/len(altezze)           # calcolo la media
    return media, min_altezze, max_altezze       # valori risultanti da ritornare a chi chiama
```

Come si chiama/esegue la funzione nel programma principale?

```
nome_funzione(valori_da_fornire)
```

In [ ]:

```
# Esempio: se abbiamo una lista di altezze
elenco_altezze = [ 170, 190, 165, 155, 186, 172, ]
```

In [ ]:

```
# ne calcoliamo la media, min e max 'chiamando' la funzione 'media_altezze' seguita
# tra parentesi () dalla lista che contiene i valori
media,m,M = media_altezze(elenco_altezze)  # e mettendo il risultato in una o più variabili
# otteniamo
media,m,M
```

NOTATE CHE

- i nomi degli argomenti che ho usato nella **definizione** della funzione sono `altezze` (che sono detti **parametri formali**), sono variabili che contengono i dati che la funzione riceve da chi la chiama
- i nomi delle variabili che ho calcolato DENTRO la funzione (min_altezze, max_altezze, somma_altezze, media) sono **variabili locali** alla funzione, esistono SOLO DENTRO la funzione e vengono eliminati quando finisce
- i nomi che ho usato nel codice che CHIAMA la funzione sono `elenco_altezze` (che sono detti **parametri attuali**), sono i dati del programma principale che effettivamente voglio fornire a questa particolare chiamata della funzione

Stile: è meglio usare nomi diversi per i parametri **formali** e per gli **attuali** per evitare di confondersi

Ancora questioni di stile: identificatori **parlanti**

- scegliete un **nome di funzione** che vi ricorda bene **cosa fa**
- scegliete i **nomi degli argomenti** in modo che sia chiaro **quali informazioni dovete fornire** alla funzione
- scegliete i **nomi delle variabili** in modo che vi sia chiaro **cosa contengono**

per cosa	GOOD	BAD
funzione	`calcola_altezza_media`	`pippo`
argomento	`altezze`	`lista`
variabile	`somma_altezze`	`sa`

Ovvero

> ***"Programma sempre come se il ragazzo che dovrà manutenere il tuo codice sia uno psicopatico violento che sa dove vivi."*** *Martin Golding*

Ciclo di vita delle variabili locali e chiamata delle funzioni

- Ogni esecuzione (chiamata della funzione) crea un **nuovo gruppo di variabili locali** della funzione (un nuovo namespace)

- inizialmente contiene solo i nomi degli argomenti (parametri formali) associati ai valori che sono stati forniti alla chiamata (parametri attuali)

- le variabili create nel corpo della funzione vengono **CREATE** in questo namespace (dopo che la funzione viene **CHIAMATA/ESEGUITA** e NON quando viene definita)

Per cui le funzioni possono anche richiamare se stesse senza problemi, **ciascuna chiamata userà un nuovo spazio di memoria personale** separato dagli spazi delle altre chiamate

Il namespace delle **variabili locali** viene **rilasciato quando si esce dalla funzione** e si torna al programma che l'ha chiamata
NOTA: la memoria che viene **rilasciata** è quella della **tabella dei nomi** (namespace).
Gli oggetti creati dalla funzione possono sopravviverle (ad esempio come valori ritornati)
MA QUESTO NON FA SPRECARE UN SACCO DI MEMORIA ???
NO, esiste un processo in background, il **garbage collector** che esamina la memoria continuamente e libera tutti gli oggetti che **non sono più raggiungibili** dai programmi in esecuzione

In [ ]:

```
## Esempio di creazione di un oggetto dentro una funzione
def concatena(prima_parte, seguito):
    'funzione che costruisce e ritorna una nuova stringa'
    nuovo_testo = 'concatenato dentro la funzione: ' + prima_parte + ' ' + seguito
    return nuovo_testo    # e la ritorno
```

In [ ]:

```
A = 'inizio'
B = 'fine'
C = concatena(A, B)       # chiamo la funzione
# stampo l'oggetto che sta nella variabile C, che è stato creato dentro la funzione
print(A, B, C, sep='\n')
```

In [ ]:

```
# proviamo a leggere la variabile 'nuovo_testo' che era DENTRO la funzione
print(nuovo_testo)
# errore perchè 'nuovo_testo' non esiste nel namespace globale
# (era nel namespace locale alla chiamata della funzione,
# che è stato disallocato uscendo)
```

Supponiamo di avere queste 3 definizioni di funzioni e il programma principale che chiama funzione1

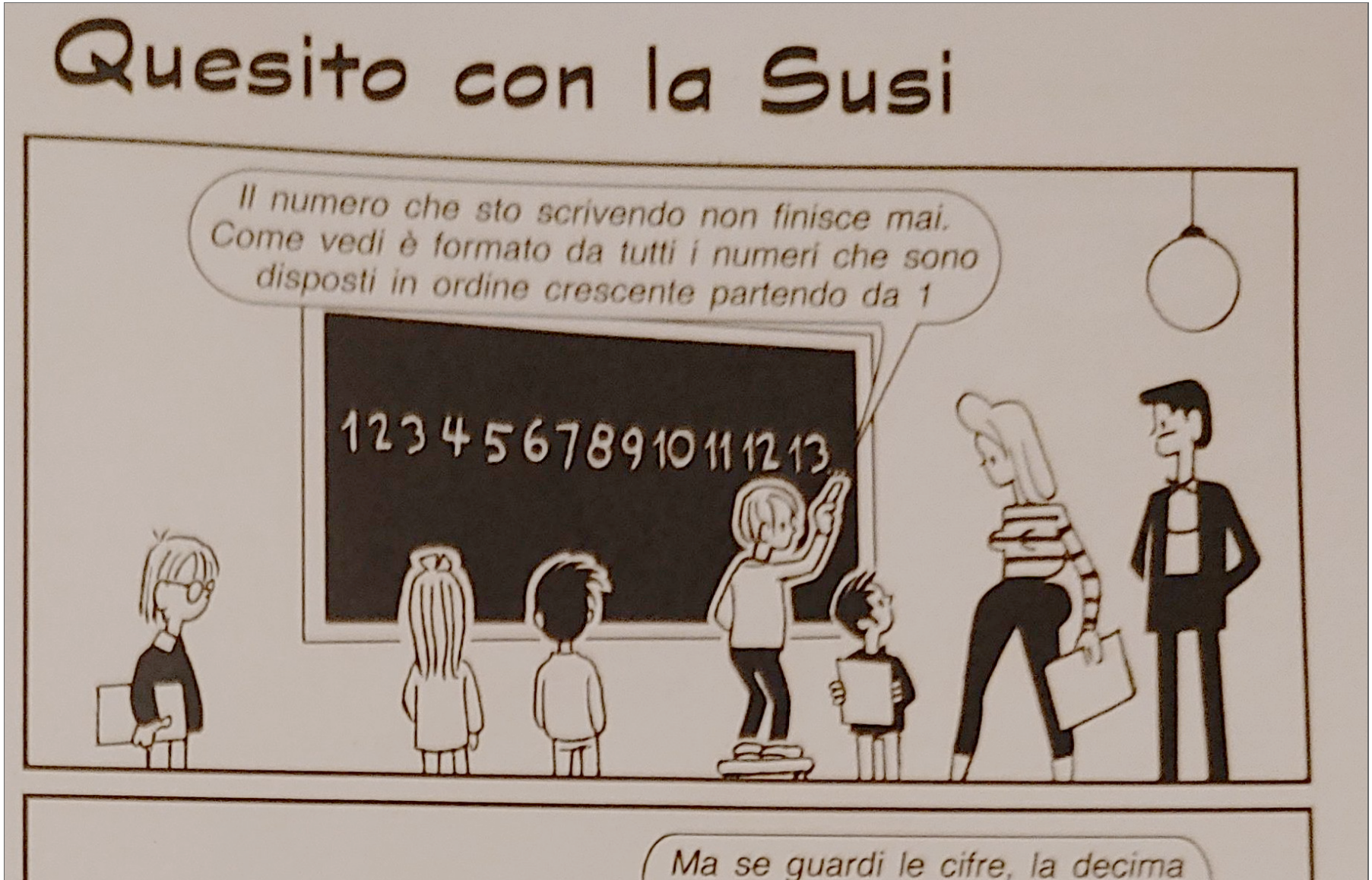
```
def funzione1(arg1,arg2)
    # ... altre istruzioni
    X = funzione2(arg1)
    ...
    return X
def funzione2(arg22)
    # ...
    Y = funzione3(arg22)
    # ...
    return Y
def funzione3(arg33)
    # ...
    Z = arg33**2
    # ...
    return Z

A,B = 1, 2
risultato = funzione1(A,B)  # programma principale
print(risultato)
```

In [ ]:

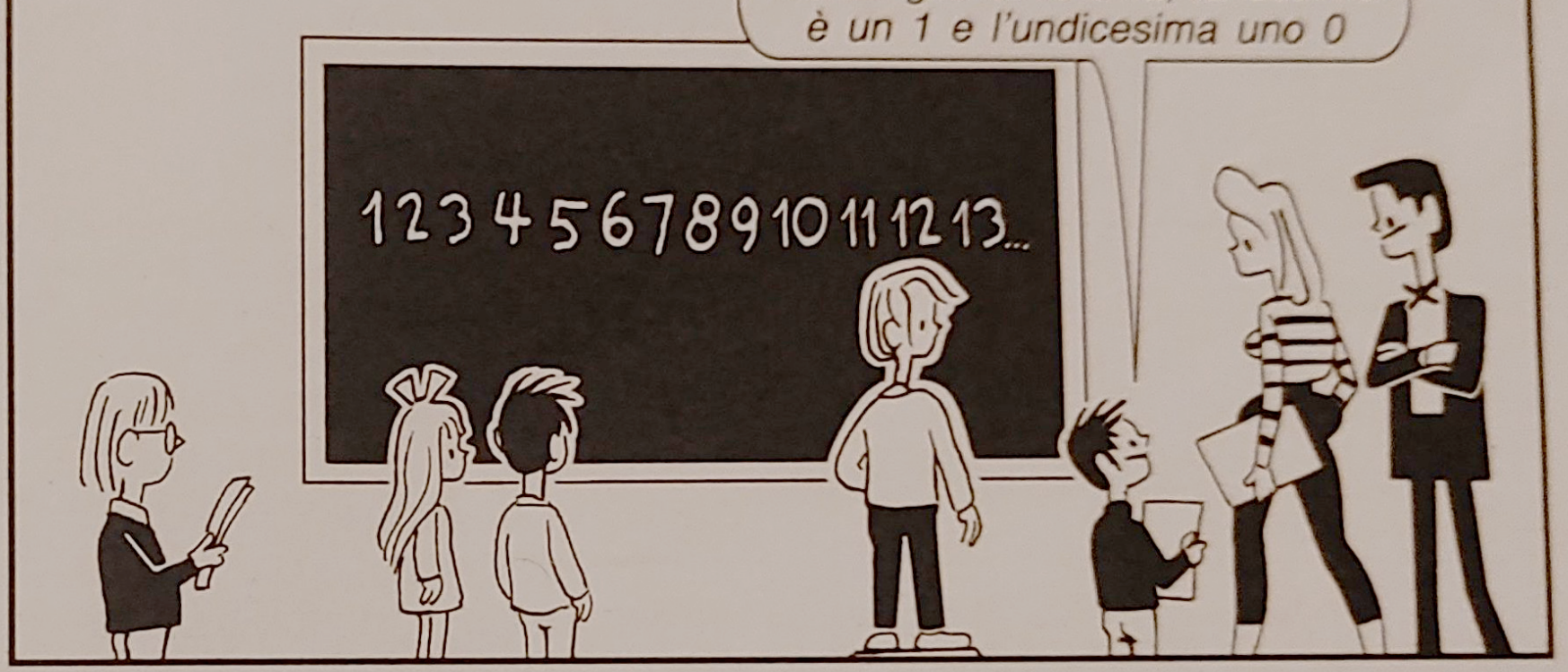
```
figura6
```

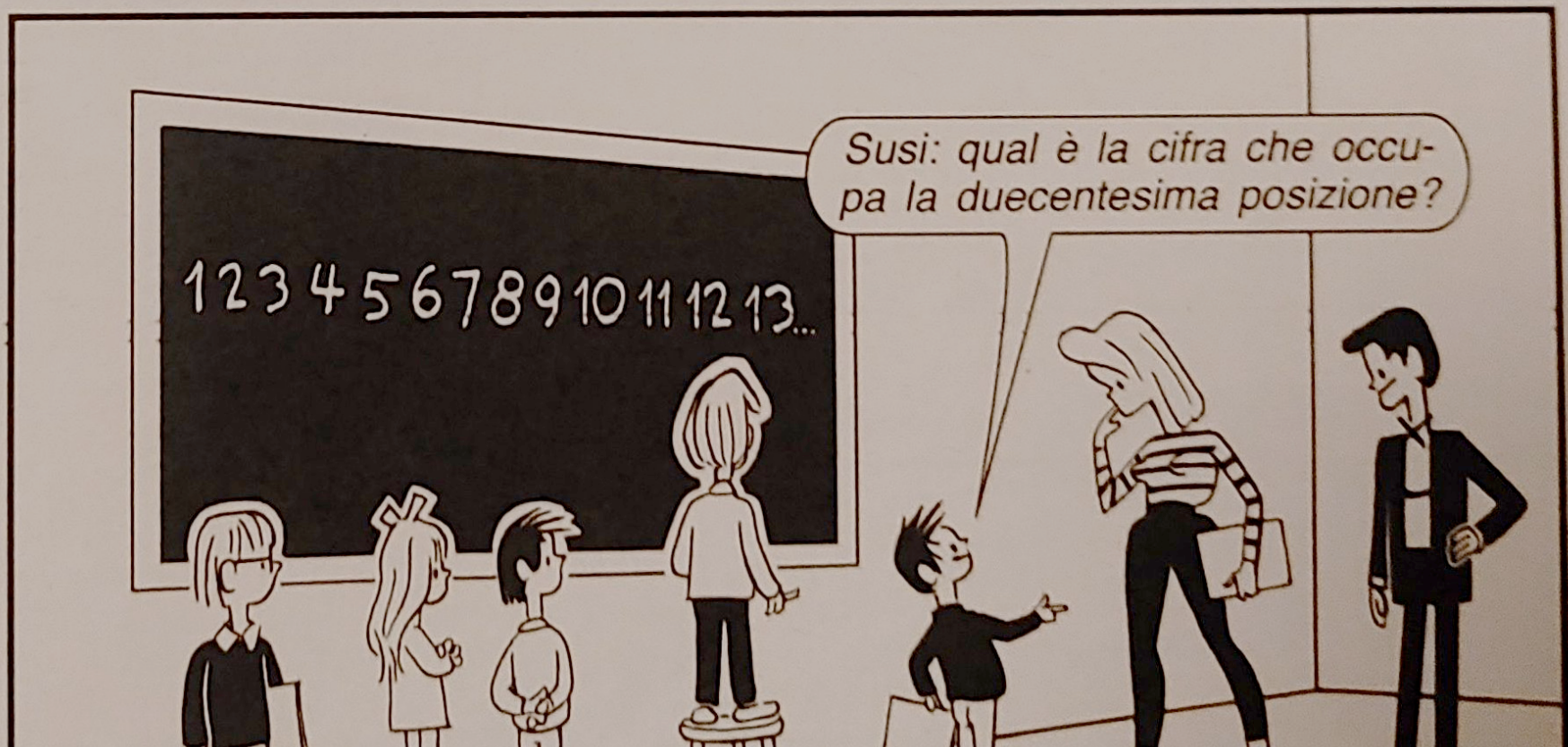
Quesito con la Susi 1

(ed ora un po' di problem-solving)



è un 1 e l'undicesima uno 0





Come analizzare il problema?

- Buttiamo giù su carta una idea anche rozza di algoritmo
- spezzando il problema in problemi più piccoli
  (così ne riduciamo la difficoltà e **semplifichiamo il debug**)
- finchè i problemi sono così piccoli da essere "ovvi"

Un primo approccio al quesito della Susi ...

costruisco la stringa e trovo il carattere!

- **Per trovare la N-esima cifra**
    - costruisco la stringa con almeno N cifre (sottoproblema più semplice)
    - ritorno il carattere a indice N-1 (ovvio)

In [ ]:

```
def cerca_con_stringa(N):
    stringona = costruisci_stringona(N)  # DA DEFINIRE
    return stringona[N-1]   # si conta a partire da 0
```

Affrontiamo il sottoproblema più semplice

- **Per costruire la stringa con almeno N cifre**
    - ripeto N volte (visto che ogni numero ha almeno una cifra)
        - concateno alla stringa corrente il prossimo intero
    - torno la stringa

Manca qualcosa?

- l'inizializzazione delle variabili! (la stringa all'inizio è vuota)

In [ ]:

```
# Per calcolare la N° cifra:
    # costruisco una stringa con le cifre
def costruisci_stringona(N):
    testo = ''                  # si parte con una stringa vuota
    for i in range(1,N+1):      # si ripete N volte a partire da 1
        testo += str(i)         # concateno le cifre del numero corrente
    return testo

len(costruisci_stringona(200)), costruisci_stringona(200)
```

Oppure posso smettere appena posso (più efficiente)

- **Per costruire la stringa con almeno N cifre**
    - ripeto se non ho raggiunto le 200 cifre
        - concateno alla stringa corrente il prossimo intero
    - torno la stringa

In [ ]:

```
# Stavolta uso un while per concatenare numeri solo se non abbiamo ancora le N cifre
# e NON so quante iterazioni mi serviranno

# Per calcolare la N° cifra:
    # costruisco una stringa con le cifre ma solo se ne mancano
def costruisci_stringona_smettendo_subito(N):
    testo = ''                  # si parte con una stringa vuota
    i = 1                       # e dal numero 1
    while len(testo) < N:       # e si continua (SOLO SE il numero di cifre è insufficiente)
        testo += str(i)         # a concatenare le cifre del numero corrente
        i += 1                  # ed a incrementarlo (VA FATTO ESPLICITAMENTE! E' UN WHILE!)
    return testo                # finito il ciclo torno la stringa

len(costruisci_stringona_smettendo_subito(200)), 'caratteri'
```

E’ una buona soluzione?

- produce un mucchio di stringhe! (una per concatenazione)
- occupando un sacco di memoria!

Seconda idea che non crea stringhe inutili

- Per calcolare la N° cifra:
    - trovo in quale numero sta la N° cifra e quante cifre sono rimaste da contare (sottoproblema più semplice)
    - tiro fuori la N° cifra dal numero trovato (ovvio)
    - la ritorno (ovvio)

In [ ]:

```
def cerca_senza_stringhe(N):
    # ritorno la coppia: numero in cui casca la cifra, e quante ne mancano
    numero, mancanti = cerca_numero_e_rimanenti(N)  # DA DEFINIRE
    testo = str(numero)              # trasformo in stringa il numero
    return testo[mancanti-1]         # e torno il carattere giusto (contando da 0)
```

- Per trovare in che numero sta la N-esima cifra
    - scandisco i numeri da 1 in poi e per ciascuno
        - trovo quante cifre ha (sottoproblema più semplice)
        - se sono di meno delle cifre ancora da contare
            - le sottraggo da quelle ancora da contare e continuo a contare (ovvio)
        - altrimenti ho raggiunto il numero cercato
            - lo ritorno assieme al numero di cifre mancanti (ovvio)

In [ ]:

```
def cerca_numero_e_rimanenti(N):
    for i in range(1,N):          # scandisco al più N-1 numeri (potrei usare il while ma finirò prima)
        cifre = num_cifre(i)      # calcolo le cifre del numero corrente
        if cifre < N:             # se sono ancora troppo poche
            N -= cifre            # le sottraggo a N e continuo il FOR
        else:                     # altrimenti siamo arrivati al numero giusto
            return i, N           # e torno il numero corrente e quante cifre mancano
                                  # return esce dal ciclo for anche se non è completo

# Notate che ho aggiornato N, che si comporta come una variabile locale
# e non modifica i valori nel programma che ci ha chiamato
```

Ma come trovo quante sono le cifre di un numero?

- opzione 1: lo trasformo in stringa e conto i caratteri (BLEAH!!!)
- opzione 2: ne calcolo il logaritmo in base 10, lo tronco a intero e sommo 1
    - il logaritmo di N è la potenza (di 10) che dà N

      (e quindi è il suo numero di cifre)
- opzione 3: genero le potenze di 10 finchè lo supero

In [ ]:

```
def opzione_1(X):
    'calcolo il numero di cifre di un numero in base 10 trasformandolo in stringa'
    return len(str(X))      # BLEAH! genero un sacco di stringhette!!!

opzione_1(100000), opzione_1(99999) # controllo che il numero salga nel punto giusto
```

In [ ]:

```
from math import log10      # uso la funzione log10 del modulo math

def opzione_2(X):
    'calcolo il numero di cifre di un numero in base 10 con i logaritmi'
    # ne calcolo il logaritmo in base 10 (è un float)
    L = log10(X)
    # lo tronco ad intero e gli sommo 1
    return int(L)+1

opzione_2(100000), opzione_2(99999) # controllo che il numero salga nel punto giusto
```

In [ ]:

```
def opzione_3(X):
    'calcolo il numero di cifre di un numero in base 10 generando le potenze'
    numcifre  = 0            # esponente iniziale
    potenza10 = 1            # 10**0 vale 1
    while potenza10 <= X:    # continuo a moltiplicare se non ho raggiunto N
        numcifre  += 1       # incremento l'esponente (numero di cifre)
        potenza10 *= 10      # aumento la potenza di 10
    return numcifre

opzione_3(100000), opzione_3(99999)  # controllo che il numero salga nel punto giusto
```

In [ ]:

```
N = 20000
# TRICK: sostituisco la funzione num_cifre con una o l'altra implementazione
num_cifre = opzione_1                       # uso la prima opzione
print(cerca_senza_stringhe(N), end=' ')
num_cifre = opzione_2                       # uso la seconda opzione
print(cerca_senza_stringhe(N), end=' ')
num_cifre = opzione_3                       # uso la terza opzione
print(cerca_senza_stringhe(N), end=' ')
```

Altre idee

N	da a	i numeri con N cifre sono	le cifre totali del blocco sono
1	1..9	9	9
2	10..99	90	180
3	100..999	900	2700
...	...	...	...
N	... .	$9 * 10^{N-1}$	$N * 9 * 10^{N-1}$

Potremmo

- contare i blocchi completi prima della N^{ma} cifra e sottrarli da N
- trovare nel primo blocco non completo il numero in cui cade la N^{ma} cifra
- trovare la posizione della cifra nel numero
- estrarre la cifra dal numero

Facciamolo a mano

- N = 200
- i due blocchi 1 cifra + 2 cifre => 9 + 180 = 189 cifre
- restano 200-189 = 11 cifre
- siamo nei numeri di 3 cifre
- 11 // 3 = 3 => quanti numeri di 3 cifre **completi** precedono quello giusto
- per cui siamo nel 4° numero di 3 cifre (100, 101, 102, **103**, ...)
- 11 % 3 = 2 => e ci serve la seconda cifra (quella a indice 1)
- '103'[1] = **'0'**

In [ ]:

```
# SOLUZIONE: il numero che contiene la 200° cifra è il 103 e mancano 2 cifre
cerca_numero_e_rimanenti(200), cerca_con_stringa(200), cerca_senza_stringhe(200),
```